Tuberías Unidireccionales (Pipes)

Malak Sanchez
Monitorias Sistemas Operativos

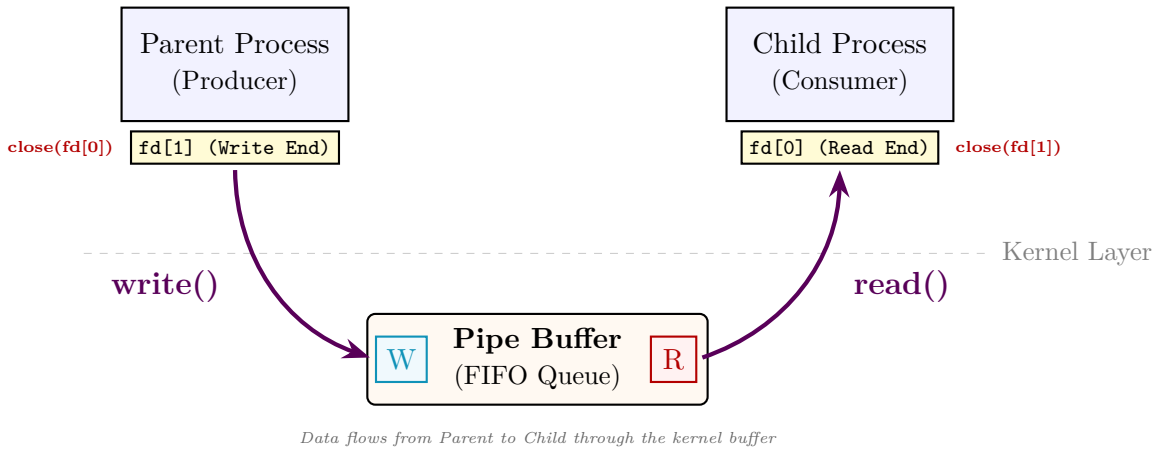
11 de octubre de 2026

1. Motivación

Gran parte del poder de la terminal de Linux reside en el operador `|`. Este permite conectar la salida de un proceso con la entrada de otro. Internamente, esto se logra mediante el mecanismo de **Pipes**.

2. Idea Fundamental

Un pipe es un buffer circular en el espacio del kernel que permite el flujo de datos en una sola dirección. Para que dos procesos se comuniquen mediante un pipe, deben tener un ancestro común que haya creado el pipe antes de realizar un `fork()`.



Data flows from Parent to Child through the kernel buffer

3. Sintaxis Básica

```c
#include <unistd.h>
int fd[2];
pipe(fd); // Creates the pipe.
          // fd[0]: Read end
          // fd[1]: Write end
```

Descripción de las operaciones

- **pipe(fd):** Crea un canal de comunicación y llena el arreglo `fd` con dos descriptores de archivo. Devuelve 0 en éxito y -1 en error.
- **write(fd[1], buf, n):** Envía `n` bytes desde el buffer `buf` hacia el pipe. Si el pipe está lleno, el proceso se bloquea.
- **read(fd[0], buf, n):** Lee hasta `n` bytes del pipe y los guarda en `buf`. Si el pipe está vacío, el proceso se bloquea hasta que haya datos.
- **close(fd):** Cierra un extremo del pipe. Es **crítico** cerrar los extremos no utilizados en cada proceso para que `read()` pueda detectar el final del archivo (EOF) cuando todos los escritores cierren el pipe.

4. Ejemplo: Comunicación Padre-Hijo

En este ejemplo, el padre envía un mensaje al hijo. Note cómo cada proceso cierra el extremo que no va a utilizar inmediatamente después del `fork()`.

```c
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    char buffer[30];

    if(pipe(fd) == -1){
        perror("pipe");
        return 1;
    }

    if(fork() > 0){ // Parent process
        close(fd[0]); // Parent will not read, close read end

        char msg[] = "Message from your parent!";
        write(fd[1], msg, strlen(msg) + 1);

        close(fd[1]); // Finished writing
        wait(NULL);   // Wait for child to finish
    }else{ // Child process
        close(fd[1]); // Child will not write, close write end

        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);

        close(fd[0]); // Finished reading
    }
    return 0;
}
```

5. Conceptos Avanzados

- **Tuberías con nombre (FIFOs):** A diferencia de los pipes anónimos, los FIFOs tienen un nombre en el sistema de archivos y permiten la comunicación entre procesos sin parentesco.

6. Ejercicios

1. Modifique el ejemplo para que el hijo responda al mensaje del padre y ambos puedan intercambiar una confirmación. ¿Cuántas tuberías anónimas necesita la comunicación dúplex?

2. Experimente qué ocurre cuando un proceso intenta leer de una tubería cuyo extremo de escritura fue cerrado por todos los procesos. Relacione el resultado con EOF.